JAMAICA REPORT

Executive Summary

This report describes the development and capabilities of the Jamaica system for sensing censors and other active network interference based on protocol classification.

Introduction

Building and testing networked applications is difficult even across standardized networks only designed to route packets from one side to the other. However, real world networks are much more complicated, particularly where security concerns lead to deployment of various types of firewalls, network monitors, "DMZs", and special protocol treatment. Data-driven design, and more particularly machine learning (ML), are increasingly used in development of these networking "middleboxes". The statistical nature of these approaches leaves a significant potential that introducing a new networked application (whether actually new or just new to the middlebox environment) could trigger erroneous behavior: e.g. an overly-sensitive ML-based firewall could label an updated HTTPS protocol as malicious.

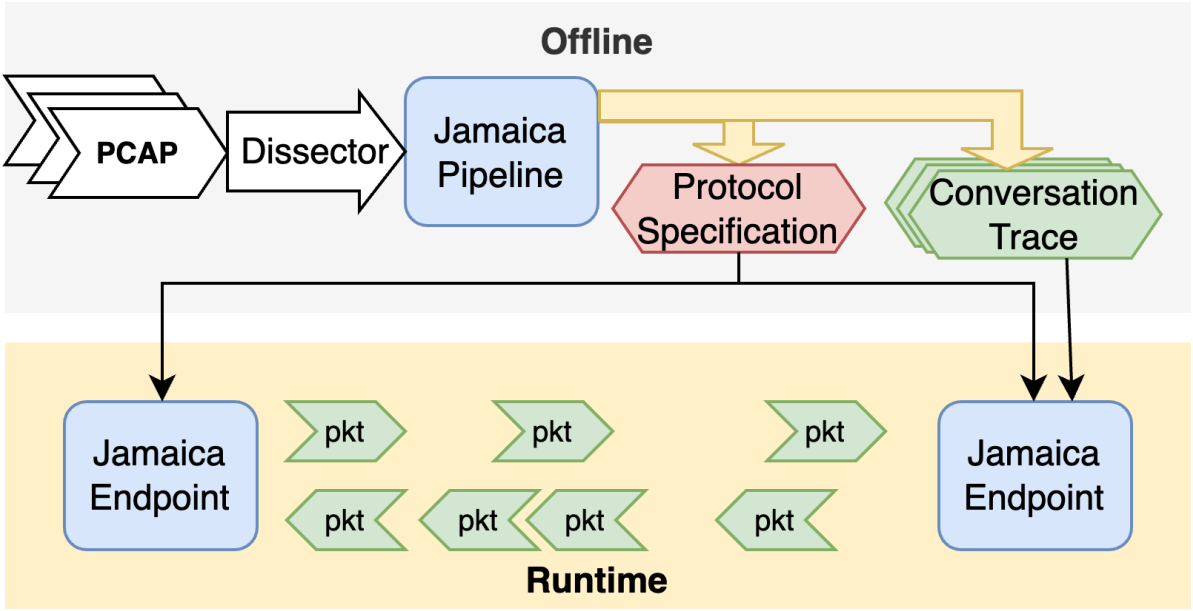


Fig. 1. The overall Jamaica system consists of an offline protocol learning pipeline and a runtime protocol mimicry application

We observe a general gap for robustly testing the environment's reaction to a networked application's traffic, short of actually running that application through the environment and seeing if it works. Even in that case, most applications provide enough resilience mechanisms that shallow observation of the application behavior will not reveal less-than-total network failure until the application is put under strain (e.g. deployed and used in-practice, rather than just initially tested). Further, there are many cases where just "deploy the application" is not a

Two Six Technologies © 2024

reasonable approach: applications might be restricted, require specific hardware components, or where the application developer wants to explore alternative protocol designs and observe their effects. The last case is particularly relevant to cases like censorship circumvention research: the researcher may observe a protocol to be censored and want to explore different permutations of that protocol that are *not* censored, before actually working through redesigning the application to produce such traffic. To fill this gap, we present *Jamaica*, which combines offline-learning and runtime-tunable network traffic generating to provide high fidelity mimicry of arbitrary application traffic.

Figure 1 illustrates the conceptual scope of Jamaica: its offline pipeline starts with packet capture (PCAP) files of traffic of interest and dissectors that provide segmentation and non-semantic labeling of individual packet payloads into fields (i.e. protocol-specific Wireshark packet dissector modules). Jamaica then performs a set of analyses on these fields to identify attributes, such as value range, entropy, and inter-field correlations. These attributes are then used to automatically generate a *protocol specification*. This consists of function specifications for generating values for fields without naively replaying prior data. Additionally, the pipeline extracts *conversation traces* from connections in the PCAP that are used as inputs for generating similar-looking network traffic.

The runtime component of Jamaica takes the *protocol specification* and a *conversation trace* and generates a bidirectional stream of traffic between two endpoints that corresponds to the overall conversation shape. Importantly, Jamaica uses a remote-control model where the conversation client transmits the conversation trace *in-band* to the server as part of the generated traffic. This enables the client to customize the conversation without pre-staging it on the server. The use of a conversation trace enables either side to detect if a network anomaly occurs that alters the packets received (or drops some entirely).

We briefly illustrate the use of Jamaica for measuring protocol filtering behaviors, such as ISPs blocking use of the Obfs4 anticensorship protocol. First, real obfs4 traffic is gathered and fed through the offline Jamaica pipeline. This produces a protocol specification for obfs4 and a set of conversation traces. Then, the Jamaica "server" endpoint is run on an uncensored cloud service and the "client" is run from inside the subject ISP. First, the deployment can measure that obfs4 is being blocked by executing the gathered conversation traces. However, the client can then try altering the mimicked obfs4 traffic to change its characteristics without transmitting the alterations out-of-band to the server; the client can even dynamically update traffic alterations in response to the network's reactions, without needing out-of-band updates for each iteration.

The Jamaica system provides the following contributions to the space of network traffic analysis, generation, and measurement:

1. Novel analysis of protocol fields based on inter-packet and inter-field correlations
2. Automatic construction of executable field generation functions for fine-grained synthesis of protocol data that captures interfield dependencies

Two Six Technologies © 2024

3. Flexible synthesis of bidirectional network traffic
4. In-band-control and error detection of measurement traffic to support minimalist measurement deployments

Background and Related Work

Jamaica is an advancement at the intersection of network traffic classification, protocol reverse engineering, and packet synthesis. We provide brief background on existing techniques in these spaces.

Network Protocols

Networked applications communicate via data packets consisting of multiple nested layers of protocols to support transmission over different levels of abstraction. For the purposes of Jamaica, we consider protocols above OSI layer 4 (i.e. transmitted on top of TCP or UDP). Examples of protocols in consideration by Jamaica are QUIC, RTP (real-time protocol), <OTHERS>. We consider protocols to principally define two components: a *packet format* and a *state machine*.

The packet format is, ideally, a regular language describing the way the bits of the protocol packet are set to represent both protocol-level information and application-level data to be transmitted. This *commonly* takes the form of a *protocol header* with protocol-level information followed by a *protocol payload* containing application data. E.g. the UDP packet format is a 4-byte header containing fixed-size entries for the source port, destination port, payload length, and checksum; the remainder of the packet is payload (i.e. application data) and its length is described by the length in the header. Packet formats are only truly constrained by the ability to write programs to construct and parse them, so they can be arbitrarily complex although best-practices encourage minimizing complexity to suit the use requirements. The overall goal of Jamaica is to automate learning the packet format of a protocol to support generating packets that conform to the format. A subpart of the packet format discovery process is *segmentation* of the protocol into discrete fields. Jamaica does *not* provide segmentation, but builds on existing tools for doing so (specifically, Wireshark dissectors, although other segmenters can be used as described below).

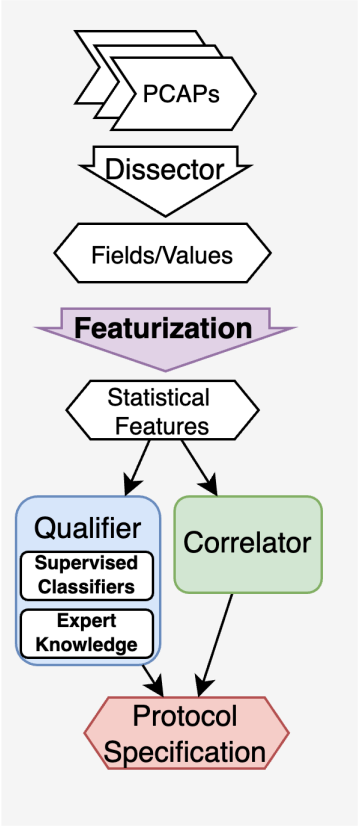
In addition to the packet format, protocols have a (implicit or explicit) state machine governing their execution. These state machines effectively provide the *semantics* that are expressed in the *syntax* of the packet format. Reverse engineering these protocol state machines, either from network traffic, program execution traces, or both, is an active area of research. However, Jamaica does *not* seek to reverse engineer the state machine of a protocol (for reasons explained in the next section).

Two Six Technologies © 2024

Traffic Classification

Network managers have a variety of motivations for understanding what applications and protocols are transiting their network and being able to differentiate treatment of packets based on that. Quality-of-service is a classic use-case, wherein time-sensitive flows like streaming video or audio could be prioritized over asynchronous data transfers. Monitoring traffic for signs of malware infection or botnet activity is another. Identifying and blocking use of censorship circumvention applications (like Tor) is also increasingly common. All of these share the task of *traffic classification* as a required capability.

Traffic classification techniques have evolved greatly from simple port-based classification to deep-packet inspection and whole-connection or even multi-connection analysis using state-of-the-art machine learning. Even in these advanced cases, classification appliances stop short of fully replicating the protocol implementation (i.e. the *protocol state machine*); they take shortcuts like analyzing individual packets, short bursts of packets (like handshakes), or looking at higher-level data like packet timings and sizes over the course of an entire connection (*traffic shape*). Depending on the case, these classification appliances then trigger actions on the network, such as (de)prioritizing traffic, queuing probes to the endpoints, or resetting the connection and blocking future connections for a time. Jamaica is aimed at mimicking arbitrary protocols to the fidelity necessary to probe classifiers to determine a) what actions they take and b) which features they use for classification.



Offline Pipeline: Protocol Analysis

Jamaica is architected as a multistage offline pipeline and a runtime application. The pipeline analyzes a corpus of segmented protocol data, extracts a protocol specification, summarizes connections into conversation traces, and compiles the protocol specification into executable generator functions.

Dissection

The first stage of the Jamaica pipeline is dissection (also known as segmentation). We treat dissection as a blackbox function that takes packet captures (PCAPs) as inputs and outputs breakdowns of each packet as a list of *fields*. Each *field* is a tuple of (field-name, start-index, end-index, value). Notably, we do not expect the dissection to provide any semantic information about the field, just to spilt the packet into syntactically correct blocks of bits representing different fields. In this implementation we use the array of open source

Two Six Technologies © 2024

dissectors created for the Wireshark/TShark project which we wrap to provide this packet-processing functionality in a rust program (via the *rtshark* library).
We note that other implementations of dissection functionality could be swapped in (or used in-addition-to) these Wireshark dissectors. E.g. recent work on automatically learning to segment various types of network protocols[TODO].

Featurization

Dissection outputs field tuples which can be associated with an individual packet, the connection that packet occurs in, and with other fields of that name. The featurization process computes statistics over the *value* element of the field tuples grouped by field-name. The statistics used include entropy, count of unique values, average, min/max, and standard deviations of both the numerical representation of the value and the length (in bits) of the value in a particular field instance (see code for full list of features). The aim of these features is to enable differentiation of fields into different *semantic types.*

Qualifier

The qualifier takes the statistical features of the fields and attempts to assign a *qualitative label* (hence "qualifier") to the field-name. Our implementation uses an ensemble of simple supervised classifiers (decision trees) and expert knowledge heuristics to assign labels from the set:

- Constant - e.g. protocol versions or reserved bits
- Sequence - e.g. protocol sequence numbers
- Encrypted - e.g. encrypted application payloads
- Length - e.g. length of payload fields
- ID - e.g. source and destination identifiers
- Flag - e.g. protocol packet type

To train the supervised classifiers, we hand label fields for the QUIC and RTP protocols and use a corpus of protocol data from them to train, then test on other protocols with no hand labeling (see Evaluation section). We expect that as protocols are analyzed, human experts can opportunistically audit the output of the qualifier and make corrections or affirm results to provide *additional* training data. Our intuition is that this process will make initial expansion of Jamaica to a new *class* of protocols (e.g. one with fields with significantly different semantics) may be more error prone, but if retrained on the data then subsequent analysis of other protocols in that class will be more accurate.

Correlator

The qualifier looked at data *across field instances* but *within field-names* for assigning a semantic label. The correlator instead attempts to infer relationships *between* field instances within the same packet. The most important intra-packet field relationships are *flag* relations and *length* relations. *Flag* relations are where the value of a *flag* field indicates (or determines, depending on your perspective on causations) the presence or absence of a set of other fields:

Two Six Technologies © 2024

e.g. a packet type field might indicate whether this packet will contain a payload field or a handshake field. *Length* relations are where the value of a *length* field describes the length of some other field. Flag correlations are identified based on boolean correlation of the value and the existence of a field; length correlations are identified (and characterized) based on a numerical correlation of the value of the length field and length of some other field. Both kinds of relationships are output for each field (if they exist) in the protocol specification.

Protocol Specification

The protocol specification is a JSON representation of the semantic attributes of the protocol to support automated synthesis of *generator functions* for each field. These are functions which can be executed by the runtime component each time a value for a certain field is needed. A snippet of this format is presented below:

```
{
  "output": "quic.length",
  "operation": "length",
  "args": [
    "quic.payload"
  ]
},
{
  "output": "quic.payload",
  "operation": "encrypted",
  "can_embed": true
},
```

In this snippet, the quic.payload field is encrypted bytes, which will trigger construction via a function providing a random byte stream. The quic.length field is a length function that takes the quic.payload field as an argument, and produces its length as the value for the length field. The *length* and *encrypted* operation names in these cases trigger specialized code generation paths for returning the numerical length of the argument in bytes and returning an encrypted stream of specified size, respectively. The format currently supports the six semantic labels as *operations* which specify a function stub family and an arbitrary number of other fields as arguments. However, only *flag* and *length* fields actually use other fields as arguments.

Conversation Trace Generator

The last piece of the offline pipeline is to generate *conversation traces* from the data corpus. Conceptually, these are representations of the raw protocol data abstracted through the protocol specification. The trace is a list of packets objects, with directionality, where each packet consists of sequential field objects specified by a field-name and either the arg(s) for the corresponding field generator function or a *raw* value to put for the field. Passing the conversation trace through the corresponding field generators produces packets with

Two Six Technologies © 2024

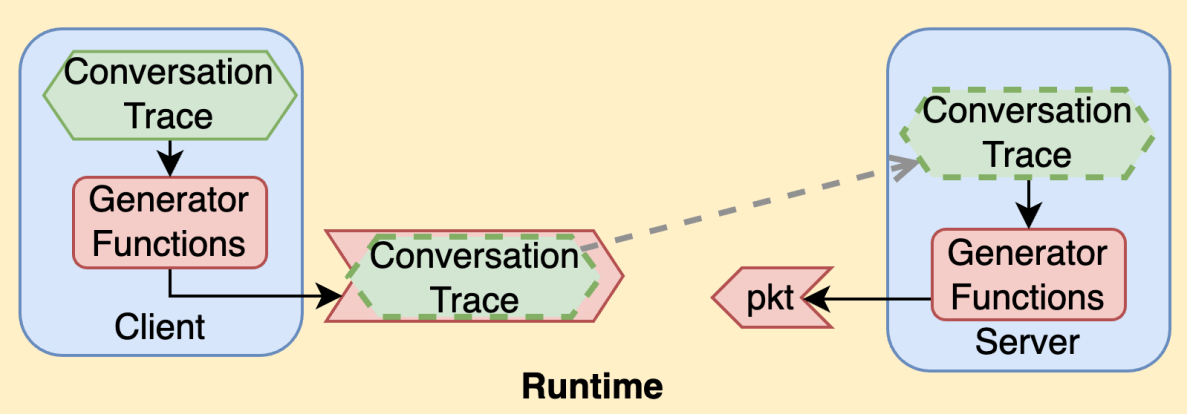
semantically-equivalent data as the original packet capture element. *However*, these fields may not have the *bitwise-equivalent* data.

Consider a QUIC packet with its *payload* and *length* fields: the conversation trace will specify a length for the *payload*, the payload generator function will generate an arbitrary string of random bytes of that length, and the length generator function will take the length of that bytestring to construct its value. Thus, the length field will have the same bits, because it represents the same payload length, but the payload field will have completely different bits. In general, Jamaica will provide arbitrary random values for *encrypted* and *ID* fields, but fields of other types are determined by structural information specified by the conversation trace.

Besides the inherent entropy from *encrypted* and *ID* fields, the conversation trace also creates a more tractable space for programmatic manipulation of the protocol data. E.g. rather than being expressed as arbitrary ranges of bits, fields are now expressible as normalized numerical arguments. These are significantly simpler and more tractable to encode in a model of conversations. We hypothesize that the corpus of existing data for a protocol, processed into conversation traces, could make a good basis for training a conversation trace *generator* model that can produce traces that retain semantic protocol correctness but sample the entire space of potential conversations more fully than gathering traffic via limited networking experiments. However, that investigation is out of the scope of this work.

Runtime Application: Traffic Generation and Measurement

The runtime application client executes a *conversation trace* in the context of a *protocol specification*, conveys the future entries of the conversation trace *in-band* to the application server, and detects deviance from expected packets that indicates network interference.



Generator Functions

The runtime application of Jamaica uses a set of *generator functions* which are compiled from the *protocol specification* to produce semantically valid values for fields, respecting inter-field

Two Six Technologies © 2024

relations and based on arguments from a *conversation trace*. We assume both the Jamaica client and server share this set of generator functions, but *do not* assume they share the entire conversation trace before runtime.

Client Conversation Execution

The client takes a conversation trace as an input, which specifies sequences of fields (and applicable arguments for generating their values) for packets for *both* directions of the conversation. The server-to-client "response" packets are tied to specific client-to-server packets, and these dependencies drive when the server chooses to send a particular packet. These dependencies also form the basis for the client detecting a network error: when it sends a packet but fails to receive the expected packet(s) back from the server, it can determine that this particular exchange of packets is where an error occurred (the packet in either or both directions being dropped). Compared to a high-level smoke test of an application, this provides more precision about *when* in a traffic stream the network begins to misbehave.

Server Conversation Execution via In-Band Control Traffic

As shown in the diagram above, the *conversation trace* originates on the client. However, the server needs *at least* the server-side of the conversation in order to respond in the manner the client expects and properly appear as the desired protocol traffic. Jamaica accomplishes this by transmitting the bulk of the conversation trace *in-band* in the client-to-server packets. Essentially, Jamaica has a unidirectional channel of control traffic to enable the client to "remote control" the server's side of the conversation, without invalidating the protocol data packets.

To preserve network-level correctness of the protocol data, Jamaica only encodes control traffic in the values of fields with *application level* semantics but no *protocol level* semantics. Generally, these are the application data payload fields (e.g. the *payload* field of QUIC packet) which have protocol semantics in terms of *length* but not in terms of actual *content* because the content is encrypted. Capitalizing on the already-high-entropy nature of these fields, Jamaica encrypts its control traffic using a stream cipher to produce similarly uniformly random data as encrypted application content. This encrypted control data is then simply substituted for the otherwise-random data of the synthesized packet field.

On the server side, the control data is extracted based on previously-received control data indicating the bitwise location(s) of encoded control data. The client thus sends two types of control data: the server side of the conversation trace to inform the server about what packets to send back, and lists of control data locations that indicate where to find additional control data in future packets. This approach creates a bootstrapping problem for the server to receive the *first* set of control data locations before it can receive any control data. To solve this, we require the server to have a small set of candidate conversation trace prefixes which it can check received client packets against. Generally only a single packet is necessary to bootstrap control data reception, so the client has full flexibility to specify the conversation trace aside

Two Six Technologies © 2024

from this initial packet. We reiterate that even in the case of this initial packet, it is only the *structure* of the packet (particularly the location of the first encrypted payload field) that needs to match the server's expectations; all the individual values in the packet are still free to be altered, subject to the protocol semantics.

Acknowledgement:

This material is based upon work supported by DARPA under Contract No. HR01123C0069. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of DARPA.

Two Six Technologies © 2024